



IIT MADRAS

DL AND GENAI PROJECT

SUBMITTED BY: TUSHAR BANSAL

ROLL NO.: 21F3001891

**MESSY MASHUP – MUSIC GENRE
CLASSIFICATION WITH NOISE**

01 ABSTRACT

This project focuses on automatic music genre classification using multi-stem audio data. The pipeline constructs synthetic training samples by mixing stems from the same genre, followed by preprocessing and augmentations such as time stretching, gain variation, and noise injection. Audio signals are converted into spectrogram representations and fed into a pretrained Audio Spectrogram Transformer (AST), fine-tuned using a two-stage strategy. Training is further stabilized using label smoothing, learning rate scheduling, and gradient clipping. This approach demonstrates that combining data-centric design with transformer-based models yields strong generalization performance.

10 Genres	0.89555 F1 Score Best: AST model	3 DL Models CNN, CRNN, AST	Synthetic Data to replicate Test Data
--------------	---	-------------------------------	--

02 INTRODUCTION

The dataset consists of multi-stem audio recordings organized by genre, where each track includes separate components such as drums, vocals, bass, and others. These stems must be combined to form complete audio signals. The dataset presents challenges such as missing stems, variability in duration, and the need for effective preprocessing and augmentation.

PROJECT OBJECTIVES

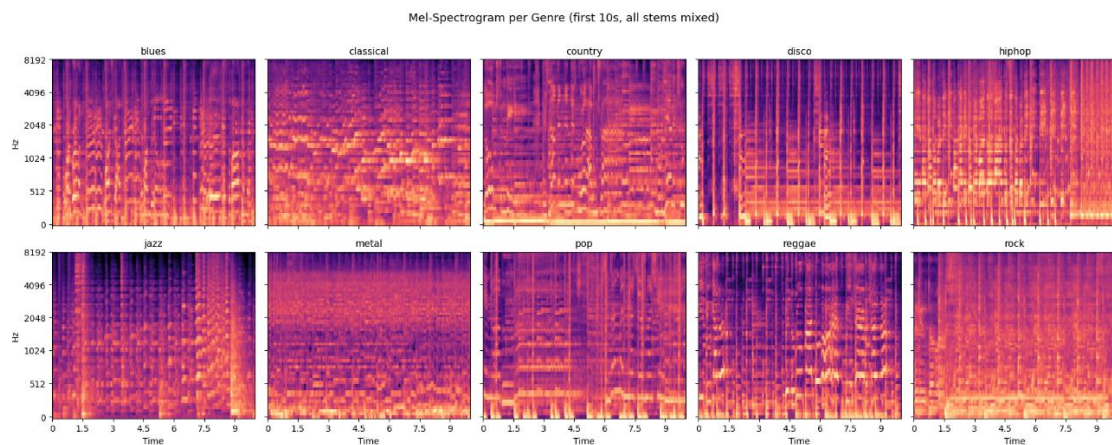
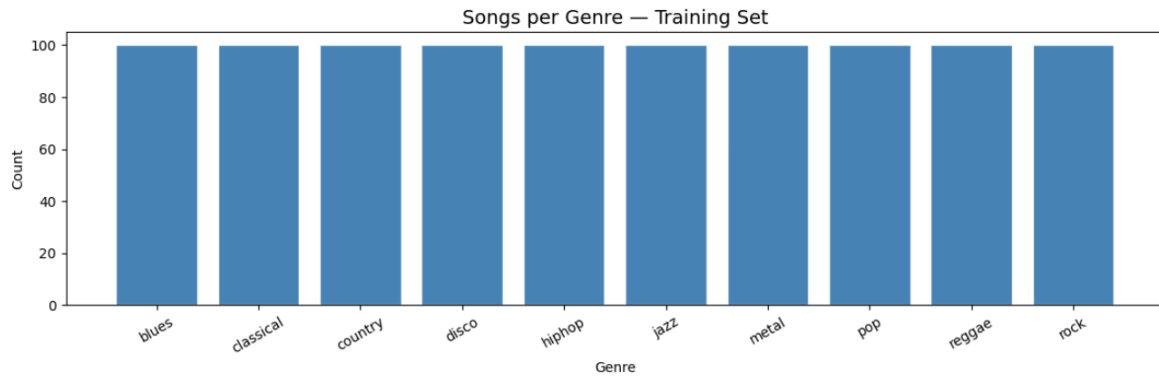
1. Develop a robust pipeline to convert multi-stem audio into meaningful representations suitable for deep learning models.
2. Leverage a pretrained transformer-based architecture (AST) to effectively capture temporal and spectral patterns in audio data.
3. Improve model generalization through data augmentation techniques and test-time augmentation strategies.
4. Achieve high classification performance on the Kaggle leaderboard while operating under computational and resource constraints.

03 DATASET AND PREPROCESSING

The dataset contains multi-stem audio recordings across 10 balanced genres, with 100 songs per genre. Each track includes drums, vocals, bass, and other stems, which are combined to form complete audio signals. An external ESC-50 dataset is used for noise augmentation, and the test set consists of variable-length mashup audio clips.

EDA AND WHAT IT REVEALS

- All classes are perfectly balanced in train dataset (ISSUE: distribution shift in test, motivation for synthetic data)
- Spectrograms show different plots per genre, can be useful (CNN, CRNN)



PREPROCESSING PIPELINE

- **Stem Mixing:** Combine available stems into a single waveform and normalize amplitude
- **Resampling:** Convert all audio to 16 kHz for consistency
- **Length Normalization:** Crop/pad audio to fixed 10-second duration
- **Feature Extraction:** Compute MFCCs, chroma, spectral and temporal features with statistical summaries
- **Augmentation:** Add environmental noise using ESC-50 dataset

WAVEFORM-LEVEL AUGMENTATIONS

Augmentation	Parameters/Range	Implementation Details
Gain Variation	Random scaling (no fixed range defined)	Applied using random amplitude scaling
Noise Injection	ESC-50 noise (random selection)	Noise added to waveform at random instances

04 FEATURE EXTRACTION

The primary feature representation is based on the **Audio Spectrogram Transformer (AST) feature extractor**, which converts raw audio waveforms into **log-mel spectrogram patches** suitable for

transformer-based modeling. This approach preserves both temporal and frequency information while enabling efficient processing through patch-based tokenization.

Unlike traditional pipelines that manually define spectrogram parameters, the AST model uses a **predefined configuration aligned with AudioSet pretraining**, ensuring compatibility with learned representations.

Parameter	Value	Rationale
Sample Rate	16,000 Hz	Matches AST pretrained configuration
Duration	10 seconds	Fixed-length input for training
Input Samples	160,000	(16k × 10s) raw waveform size
Feature Type	Log-Mel Spectrogram	Captures perceptually relevant frequency features
n_mels	128	Standard mel resolution for AST
Patch Size	16 × 16	Converts spectrogram into tokens
Input Shape	(1, 128, 1024) approx.	Spectrogram representation before patching

05 MODEL AND TRAINING

Input Representation

Two input pipelines were used:

- **Classical ML:** Handcrafted feature vectors (~300+ dimensions) extracted from audio
- **Deep Learning:** Log-mel spectrograms (128 mel bins) and AST waveform inputs (160,000 samples)

MODEL 1 – GRADIENT BOOSTING MODELS (CLASSICAL ML)

Handcrafted features (~300+ dimensions) were extracted using:

FCC + delta → **160 features**, Chroma → **24 features**, Mel energy (64 bands) → **128 features**, Spectral contrast → **14 features**, Spectral + temporal features + tempo

Models:

LightGBM: n_estimators = 500, learning_rate = 0.05, max_depth = 7, num_leaves = 63

XGBoost: n_estimators = 500, learning_rate = 0.05, max_depth = 7

CatBoost: iterations = 500, learning_rate = 0.05, depth = 8 (GPU training)

All ML models use **80–20 train-validation split** with macro F1 evaluation.

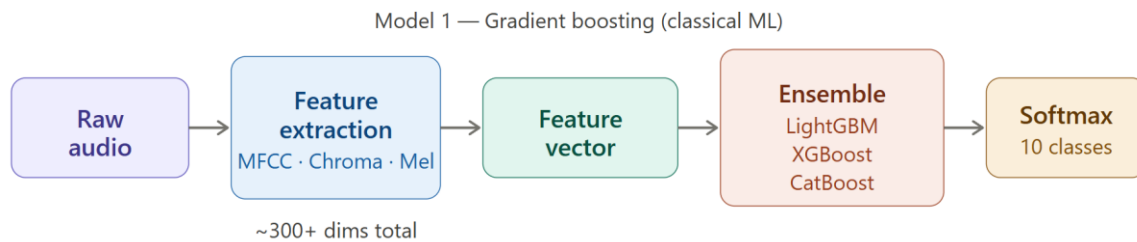


Figure 1: End-to-end inference pipeline for the gradient boosting model.

MODEL 2 — CNN (FROM SCRATCH)

Input: Mel-spectrogram: (1, 128, T), Sample rate: 22,050 Hz, Duration: 20 seconds, FFT: 2048, Hop: 512

Architecture: 4 Conv blocks: Channels: 1 → 32 → 64 → 128 → 256

Each block: Conv → BN → ReLU → Conv → BN → ReLU → MaxPool

Dropout: 0.15 → 0.25, AdaptiveAvgPool → FC: 256 → 128 → 10

Training: Epochs: 20, Batch size: 16, Optimizer: AdamW (lr = 3e-4), Loss: CrossEntropy + label smoothing (0.1), Scheduler: Cosine Annealing, Early stopping (patience = 5)

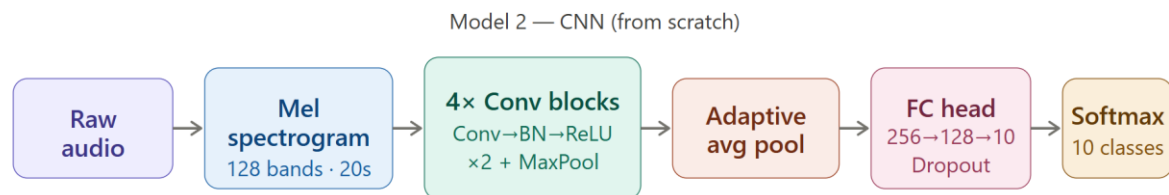


Figure 2: End-to-end inference pipeline for the CNN model.

MODEL 3 — CRNN (CNN + BiLSTM)

Architecture: CNN frontend: 4 Conv blocks → output channels 256, Temporal modeling:

Bidirectional LSTM Hidden size: 256, Layers: 2, Attention pooling over time, Fully connected head: 512 → 256 → 10

Training: Epochs: 20, Batch size: 16, Optimizer: AdamW (lr = 3e-4), Label smoothing (0.1), Cosine LR scheduler, Early stopping

CRNN captures long-range temporal dependencies missing in CNNs.

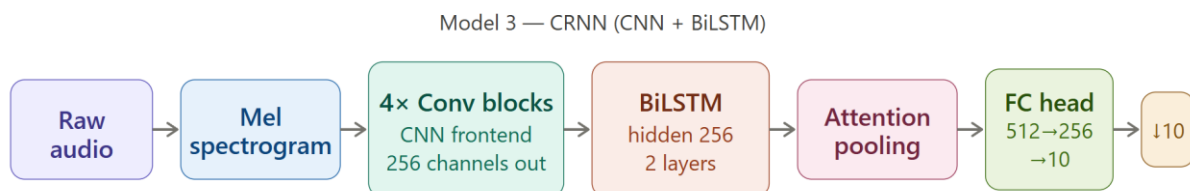


Figure 3: End-to-end inference pipeline for the CRNN model.

MODEL 4 – AUDIO SPECTROGRAM TRANSFER (AST)

Pretrained Model: MIT/ast-finetuned-audioset-10-10-0.4593, Pretrained on AudioSet (2M clips, 527 classes)

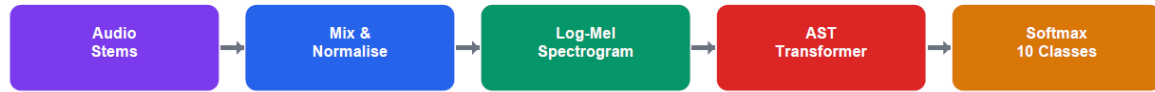


Figure 1: End-to-end inference pipeline for the best-performing AST model.

Input: Raw waveform: $16\text{ kHz} \times 10\text{s} = 160,000\text{ samples}$

Key Design: Synthetic mashup training (matches test distribution), Transformer processes spectrogram patches internally.

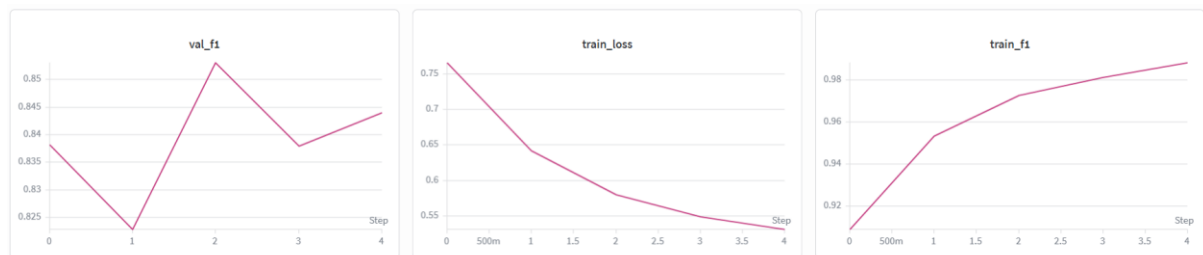
Training Strategy (AST)

Phase 1 — Classifier Warmup: Backbone frozen, Epochs: 3, Learning rate: $3\text{e-}4$

Phase 2 — Full Fine-Tuning: All layers unfrozen, Epochs: 5, Learning rates: Backbone: $2\text{e-}5$, Head: $2\text{e-}4$ ($10\times$)

Training Configuration: Batch size: 6, Gradient accumulation: 2, Weight decay: 0.01, Loss:

CrossEntropy + label smoothing (0.1), Scheduler: Cosine with warmup, Gradient clipping: 1.0, Train samples: 5000 synthetic mashups, Train/Val split: 85% / 15%



Wandb charts for ast-phase-2

Model	Type	Input	Key Architecture	Key Hyperparameters	Strength
LightGBM	Classical ML	~300+ feature vector	Gradient boosting trees	n_estimators=500, lr=0.05, max_depth=7	Fast, strong tabular baseline
XGBoost	Classical ML	~300+ feature vector	Gradient boosting trees	n_estimators=500, lr=0.05, max_depth=7	Robust and stable performance
CatBoost	Classical ML	~300+ feature vector	Ordered boosting	iterations=500, depth=8, GPU	Handles feature interactions well
GenreCNN	Deep Learning (CNN)	(1, 128, T) spectrogram	4 Conv blocks (1→256) + FC	lr=3e-4, batch=16, epochs=20	Captures local spectral patterns
CRNN (BiLSTM)	Deep Learning (Hybrid)	(1, 128, T) spectrogram	CNN + BiLSTM (hidden=256, layers=2) + attention	lr=3e-4, batch=16, epochs=20	Models temporal dependencies

AST Transformer	Pretrained Transformer	Raw audio (160k samples)	Audio Spectrogram Transformer	lr=3e-4 → 2e-5, batch=6, epochs=3+5	Best performance via transfer learning
------------------------	------------------------	--------------------------	-------------------------------	-------------------------------------	--

06 RESULTS AND ANALYSIS

All models were trained with metrics logged using **Weights & Biases (W&B)** (project: 21f3001891-t12026). The primary evaluation metric is **macro F1-score**, which treats all 10 genres equally and aligns with the Kaggle evaluation criterion.

Model	Type	Val Macro F1	Key Characteristics
LightGBM	Classical ML	0.7145	Fast, feature-based learning
XGBoost	Classical ML	0.7371	Robust boosting model – but poorly performed at testing because of randomized test dataset
CatBoost	Classical ML	0.7172	GPU training, strong tabular learning
GenreCNN	CNN (scratch)	0.4892	Local spectral feature learning
CRNN (BiLSTM)	CNN + RNN	0.6915	Temporal sequence modeling
AST Transformer	Pretrained Transformer	0.8440	Transfer learning + synthetic data

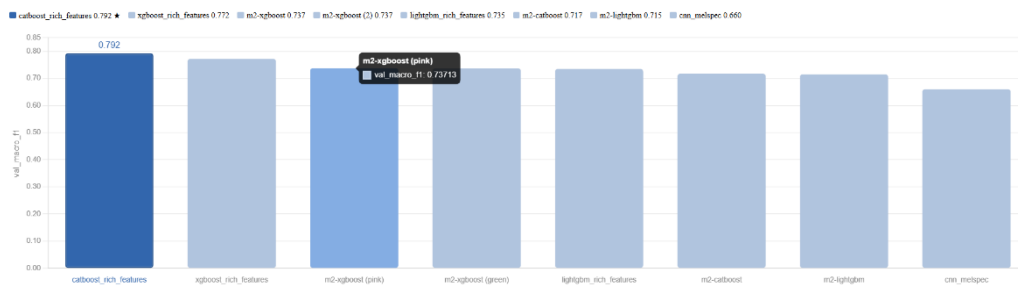


Fig 1: overfitting models

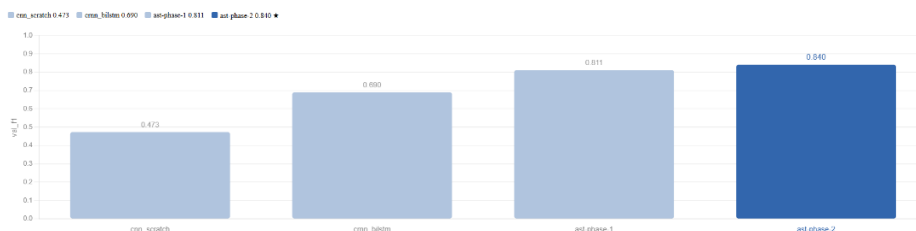


Fig 2: submission models

KEY OBSERVATIONS

AST Transformer — Best Performance

- Achieved the highest validation and Kaggle performance (**0.88931 F1**)
- Pretraining on **AudioSet (2M samples)** enabled strong feature transfer

- **Two-phase fine-tuning + TTA (5 crops)** significantly improved generalization
- Synthetic mashup training effectively reduced the train-test distribution gap

CRNN (CNN + BiLSTM)

- Outperformed CNN by modeling **temporal dependencies**
- Bidirectional LSTM captured sequential patterns (rhythm, progression)
- However, increased complexity did not match transformer performance

GenreCNN (From Scratch)

- Learned meaningful **local time-frequency features**
- Performance limited due to: Lack of long-range modelling, Insufficient data for deep training from scratch

07 CONCLUSION

This project demonstrates that combining data-centric design with pretrained transformer models yields strong performance in audio classification. Synthetic mashup training effectively bridges distribution gaps, while AST fine-tuning and test-time augmentation significantly improve generalization. The results highlight the importance of transfer learning and efficient optimization under practical computational constraints.

CHALLENGES FACED

The primary challenge was the **train/test distribution mismatch** — models trained on individual stems consistently underperformed on the mashup test set until synthetic mashup augmentation was introduced. **GPU memory** constraints required gradient accumulation (effective batch=12) and careful mixed-precision training to fit the 86M-parameter AST. The **missing stems** issue (some songs lacking vocals or bass) required defensive file-existence checks throughout the pipeline.

08 FUTURE SCOPE

1. **K-fold cross-validation:** With only 1000 training songs, a single 85/15 split is noisy. K-fold would give more reliable estimates.
2. **Better augmentation:** Pitch shifting, more aggressive tempo changes (currently $\pm 12\%$), harmonic distortion.
3. **Ensemble:** Combine predictions from CNN, CRNN, and AST — averaging their softmax probabilities often beats any single model.
4. **Larger pretrained models:** Try MERT (Music Encoder Representations from Transformers) specifically pretrained on music, or larger AST variants.
5. **Increase TTA:** More crops ($n_{tta}=10$ or 20) for more stable inference.

08 REFERENCES

1. Gong, Y., Chung, Y.-A., & Glass, J. — *AST: Audio Spectrogram Transformer* (2021)
2. Hugging Face Transformers — AST model implementation (MIT/ast-finetuned-audioset)
3. AudioSet — Large-scale dataset for audio pretraining